

Criptografia homomórfica sobre os inteiros

Do esquema DGHV parcialmente homomórfico ao bootstrapping

Gonalo Soares

Universidade do Minho, Departamento de Informtica
a108393@alunos.uminho.pt

Abstrato. Este documento apresenta a ideia de criptografia homomórfica e desenvolve o esquema de cifra sobre os inteiros proposto por van Dijk, Gentry, Halevi e Vaikuntanathan. Comea-se pelo esquema parcialmente homomórfico DGHV, cuja segurana  relacionada com o problema do mximo divisor comum aproximado, e mostra-se por que razo a soma e a multiplico de ciphertexts induzem, respetivamente, XOR e AND sobre bits. Analisa-se depois o crescimento do rudo, a noo de esquema bootstrappable introduzida por Gentry e a transformao de *squashing* usada pelo DGHV para reduzir a complexidade do circuito de decifrao. O desenvolvimento terico  acompanhado por duas implementaoes educativas em Python. Estas reproduzem a estrutura dos algoritmos, mas utilizam parmetros pequenos e no devem ser interpretadas como implementaoes criptograficamente seguras.

Palavras-chave: criptografia homomórfica, DGHV, bootstrapping, rudo, mximo divisor comum aproximado.

Índice

1. Introdução	3
2. Homomorfismo e cifra homomórfica	3
3. O esquema DGHV parcialmente homomórfico	3
3.1. Parâmetros e notação	3
3.2. Geração de chaves	4
3.3. Cifra e decifração	4
3.4. Por que razão a implementação é homomórfica	4
3.5. Exemplo numérico	5
3.6. Avaliação de circuitos e crescimento do ruído	5
3.7. Hipótese de segurança	6
4. De parcialmente a totalmente homomórfico	6
4.1. A ideia de bootstrapping	6
5. Decifração esmagada no DGHV	6
5.1. Novos parâmetros e nova chave secreta	6
5.2. Estrutura de uma cifra esmagada	7
5.3. Decifração e condições de correção	8
6. Pipeline do bootstrap implementado	9
6.1. Chave de bootstrapping	9
6.2. Contagem dos bits por coluna	9
6.3. Soma, arredondamento e resultado	9
6.4. Avaliação do circuito de teste	10
7. Limitações da implementação	10
8. Conclusão	10
Referências	11
A. Notebook parcialmente homomórfico	12
B. Notebook com decifração esmagada e bootstrap	16

1. Introdução

O armazenamento e o processamento de informação em infraestruturas externas obrigam frequentemente o utilizador a confiar dados sensíveis a uma entidade que não controla. Uma cifra convencional protege esses dados enquanto estão armazenados ou em trânsito, mas, em geral, exige a sua decifração antes de qualquer cálculo. A criptografia homomórfica procura alterar esta situação: pretende-se aplicar um circuito diretamente a ciphertexts e obter outro ciphertext que, quando decifrado, revele o resultado correto do circuito.

A primeira construção de uma cifra totalmente homomórfica foi apresentada por Gentry através de reticulados ideais [1]. O contributo conceptual central dessa construção é o *bootstrapping*: se um esquema conseguir avaliar homomorficamente uma versão ligeiramente aumentada do seu próprio circuito de decifração, então pode ser transformado num esquema capaz de avaliar circuitos de profundidade arbitrária. Pouco depois, van Dijk, Gentry, Halevi e Vaikuntanathan propuseram uma construção baseada apenas em operações sobre inteiros [2]. Esta segunda construção, habitualmente designada por DGHV, é especialmente adequada para compreender os mecanismos fundamentais, embora os seus parâmetros práticos sejam muito grandes.

O presente texto acompanha a progressão do artigo DGHV [2]: apresenta primeiro um esquema parcialmente homomórfico, estuda a sua correção e o crescimento do ruído, e desenvolve depois a transformação que torna o circuito de decifração avaliável homomorficamente. As noções algébricas de homomorfismo são desenvolvidas com maior detalhe em *Notas sobre Homomorfismos* [3].

2. Homomorfismo e cifra homomórfica

Sejam duas estruturas com operações correspondentes e uma aplicação h entre os respectivos conjuntos suporte. A aplicação é um homomorfismo quando preserva essas operações. Para uma operação binária, a condição abstrata escreve-se

$$h(f_{A(a,b)}) = f_{B(h(a),h(b))}.$$

Na criptografia, é útil imaginar o espaço das mensagens e o espaço dos ciphertexts como as duas estruturas. Contudo, as cifras modernas são aleatórias: duas chamadas a Encrypt sobre a mesma mensagem produzem normalmente ciphertexts diferentes. Por essa razão, não se exige a igualdade literal

$$\text{Encrypt}(f(m_1, m_2)) = \text{Evaluate}(f, \text{Encrypt}(m_1), \text{Encrypt}(m_2)).$$

A condição de correção relevante é a igualdade após decifração:

$$\text{Decrypt}(\text{sk}, \text{Evaluate}(\text{pk}, C, c_1, \dots, c_t)) = C(m_1, \dots, m_t),$$

onde c_i cifra m_i . Esta condição preserva o significado da operação, ainda que o ciphertext obtido por avaliação não seja idêntico a uma nova cifra do mesmo resultado. Esta distinção complementa a apresentação mais algébrica feita nas notas do autor [3].

Um esquema homomórfico de chave pública é descrito pelos algoritmos

$$\mathcal{E} = (\text{KeyGen}, \text{Encrypt}, \text{Evaluate}, \text{Decrypt}).$$

KeyGen gera as chaves; Encrypt cifra uma mensagem; Evaluate aplica um circuito aos ciphertexts; e Decrypt recupera o resultado. Um esquema é parcialmente ou *somewhat* homomórfico quando suporta apenas uma classe limitada de circuitos, tipicamente devido ao crescimento de um termo de erro. É totalmente homomórfico quando suporta circuitos arbitrários, mantendo compactos o ciphertext e a decifração [1].

3. O esquema DGHV parcialmente homomórfico

3.1. Parâmetros e notação

O esquema da Secção 3 do artigo DGHV [2] cifra bits $m \in \{0, 1\}$. Os parâmetros principais são:

- η , número aproximado de bits da chave secreta p ;
- γ , número aproximado de bits dos elementos da chave pública;
- ρ , tamanho do ruído nos quase-múltiplos públicos de p ;

- ρ' , tamanho do ruído usado na cifra;
- τ , quantidade de elementos adicionais da chave pública.

No notebook parcialmente homomórfico, estes parâmetros aparecem na classe `Parametros`. As classes `SecretKey`, `PublicKey`, `KeyPair` e `Ciphertext` representam diretamente os objetos manipulados pelos quatro algoritmos do esquema. A versão completa do notebook encontra-se no Anexo A.

3.2. Geração de chaves

A chave secreta é um inteiro ímpar p com aproximadamente η bits. Para cada índice i , gera-se um quase-múltiplo de p :

$$x_i = pq_i + r_i,$$

onde q_i é grande e o ruído r_i é pequeno. Depois de ordenar os valores, x_0 é escolhido como o maior. O algoritmo reinicia se x_0 não for ímpar ou se o seu ruído não for par, exatamente como indicado no artigo DGHV [2].

No código, `random_impair` gera p e `quase_multiplo` produz os valores x_i . O método `DGHV.keygen` guarda p em `SecretKey` e $(x_0, x_1, \dots, x_\tau)$ em `PublicKey`.

3.3. Cifra e decifração

Para cifrar $m \in \{0, 1\}$, escolhem-se um ruído pequeno r e um subconjunto aleatório $S \subseteq \{1, \dots, \tau\}$. A cifra é

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0},$$

onde os parênteses com índice representam a redução para um representante próximo de zero módulo x_0 . Como cada x_i é próximo de um múltiplo de p , o ciphertext pode ser reescrito conceptualmente como

$$c = pq + 2R + m,$$

para certos inteiros q e R . A decifração é então

$$m' = (c \bmod p) \bmod 2.$$

Se o ruído $2R$ permanecer suficientemente pequeno, a redução módulo p elimina o termo pq e a redução módulo 2 elimina o ruído par, restando m . No programa, esta construção corresponde aos métodos `DGHV.enc` e `DGHV.dec` de `somewhat_homomorphic.py` e do primeiro notebook.

3.4. Por que razão a implementação é homomórfica

Sejam duas cifras corretas

$$c_1 = pq_1 + 2r_1 + m_1$$

e

$$c_2 = pq_2 + 2r_2 + m_2.$$

Ao somá-las obtém-se

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2).$$

Depois de reduzir módulo p e módulo 2,

$$\text{Decrypt}(c_1 + c_2) = (m_1 + m_2) \bmod 2 = m_1 \oplus m_2.$$

Por isso, a soma de inteiros implementada em `DGHV.soma` corresponde a uma porta XOR no espaço das mensagens.

No produto,

$$c_1 c_2 = (pq_1 + 2r_1 + m_1)(pq_2 + 2r_2 + m_2).$$

Todos os termos, exceto $m_1 m_2$, contêm um fator p ou um fator 2. Logo, para certos inteiros Q e R ,

$$c_1 c_2 = pQ + 2R + m_1 m_2,$$

e, enquanto o novo ruído for pequeno,

$$\text{Decrypt}(c_1 c_2) = m_1 m_2 = m_1 \wedge m_2.$$

Assim, `DGHV.multiplicacao` implementa uma porta AND. A propriedade homomórfica não resulta apenas do facto de o programa somar ou multiplicar números; resulta da forma algébrica $pq + 2r + m$, que é preservada por essas operações.

3.5. Exemplo numérico

Considere-se apenas para ilustração $p = 1009$. Escolhem-se duas cifras com ruído pequeno:

$$c_1 = 4 \cdot 1009 + 2 \cdot 3 + 1 = 4043$$

e

$$c_2 = 3 \cdot 1009 + 2 \cdot 2 + 0 = 3031.$$

As mensagens são recuperadas por

$$(4043 \bmod 1009) \bmod 2 = 7 \bmod 2 = 1$$

e

$$(3031 \bmod 1009) \bmod 2 = 4 \bmod 2 = 0.$$

Para a soma,

$$(4043 + 3031) \bmod 1009 = 11, \quad 11 \bmod 2 = 1,$$

que coincide com $1 \oplus 0 = 1$. Para o produto, basta observar que $4043 \bmod 1009 = 7$ e $3031 \bmod 1009 = 4$, pelo que

$$(4043 \cdot 3031) \bmod 1009 = 7 \cdot 4 = 28, \quad 28 \bmod 2 = 0,$$

coincidindo com $1 \wedge 0 = 0$. Estes números não são seguros; servem apenas para mostrar a preservação das operações.

3.6. Avaliação de circuitos e crescimento do ruído

O método `DGHV.evaluate` lê um circuito em notação pós-fixa. O token `ADD` retira duas cifras da pilha e chama soma `MULT` chama `multiplicacao`. Por exemplo,

```
a b ADD c MULT
```

representa $(a \oplus b) \wedge c$.

A soma aumenta o ruído de forma aproximadamente aditiva, enquanto a multiplicação cria produtos entre os termos de ruído. Assim, cada ciphertext tem uma espécie de orçamento de ruído: enquanto o erro estiver muito abaixo de p , a redução módulo p continua a recuperar o representante correto; quando o erro se aproxima de $\frac{p}{2}$, a redução pode escolher o representante errado e a paridade final deixa de corresponder à mensagem original.

Os parâmetros controlam diretamente este equilíbrio. O parâmetro ρ' define o tamanho do ruído introduzido em cada vez que algo é cifrado, enquanto ρ controla o ruído dos quase-múltiplos usados na chave pública. Por outro lado, η controla o tamanho da chave secreta p , isto é, a margem disponível antes de o ruído se tornar perigoso. Escolher ρ ou ρ' demasiado grandes faz com que as cifras comecem já com muito ruído; escolher η demasiado pequeno faz com que p seja pequeno e, portanto, que haja pouco espaço para acumular ruído. Em ambos os casos, poucas multiplicações podem bastar para quebrar a correção da decifração.

Esta é a razão pela qual esta primeira construção é apenas parcialmente homomórfica [2]. O primeiro notebook inclui um circuito com várias multiplicações para tornar este limite observável, embora o resultado concreto varie porque a geração de chaves é aleatória.

3.7. Hipótese de segurança

Um atacante conhece vários valores públicos da forma $x_i = pq_i + r_i$, mas não conhece p , q_i ou os pequenos erros r_i . Recuperar p a partir destes quase-múltiplos é uma versão aproximada do problema do máximo divisor comum. Se os erros fossem nulos, calcular o máximo divisor comum dos x_i revelaria imediatamente p ; os erros impedem esse cálculo direto.

O artigo DGHV [2] relaciona a segurança semântica do esquema parcialmente homomórfico com uma variante decisional do problema do máximo divisor comum aproximado. Esta redução não significa que qualquer escolha de inteiros seja segura. Os tamanhos relativos de η , γ , ρ , ρ' e τ são parte essencial da hipótese. Os parâmetros pequenos dos notebooks destinam-se apenas a tornar a execução possível e não oferecem segurança criptográfica.

4. De parcialmente a totalmente homomórfico

4.1. A ideia de bootstrapping

Gentry [1] observou que uma cifra quase sem orçamento de ruído poderia passar por um *refresh* se fosse possível decifrá-la e voltar a cifrar o resultado. Fazer isso diretamente revelaria a mensagem. O *bootstrapping* evita essa revelação ao avaliar homomorficamente o circuito de decifração, usando uma versão cifrada da chave secreta [1].

Se D_E for o circuito de decifração do esquema E , a operação conceptual é

$$c_{\text{novo}} = \text{Evaluate}(\text{pk}, D_E, \text{Encrypt}(\text{pk}, \text{sk}), c_{\text{antigo}}).$$

O resultado cifra

$$\text{Decrypt}(\text{sk}, c_{\text{antigo}}),$$

mas a mensagem nunca aparece em claro durante a avaliação. Um esquema é *bootstrappable* quando consegue avaliar o seu circuito de decifração e uma pequena operação adicional. O teorema de Gentry [1] permite então construir um esquema totalmente homomórfico a partir desse esquema.

O DGHV original [2] decifra calculando aproximadamente $\frac{\epsilon}{p}$. Esse cálculo ainda é demasiado complexo para a profundidade suportada pelo próprio esquema. A Secção 6 do artigo DGHV [2] usa por isso a transformação chamada *squashing the decryption circuit*, que substitui a divisão por uma soma mais simples.

5. Decifração esmagada no DGHV

5.1. Novos parâmetros e nova chave secreta

Introduzem-se três parâmetros:

- κ , precisão da aproximação do inverso de p ;
- θ , número de posições não nulas da nova chave secreta;
- Θ , comprimento total dessa chave, chamado `big_theta` no notebook.

O artigo DGHV [2] escolhe,

$$\kappa = \frac{\gamma\eta}{\rho'}.$$

O notebook calcula esse valor e impõe ainda uma pequena margem relativamente a γ . Gera-se um vetor binário com poucos 1 e muitos 0

$$s = (s_1, \dots, s_{\Theta}), \quad \sum_i s_i = \theta.$$

Em seguida, define-se

$$x_p = \left\lfloor \frac{2^\kappa}{p} \right\rfloor$$

Este valor não representa diretamente o inverso de p . Ele é o numerador inteiro de uma aproximação com denominador 2^κ :

$$\frac{x_p}{2^\kappa} \approx \frac{1}{p}.$$

Ou seja, κ indica quantos bits de precisão são usados para guardar a aproximação racional do inverso de p sem recorrer a números de vírgula flutuante.

Depois escolhem-se inteiros $u_i \in [0, 2^{\kappa+1})$ que satisfazem

$$\sum_i s_i u_i = x_p \bmod 2^{\kappa+1}.$$

Finalmente, define-se $y_i = \frac{u_i}{2^\kappa}$. O que se obtém é a propriedade central do *squashing*: a soma dos y_i seleccionados pelas posições onde $s_i = 1$ aproxima o inverso de p :

$$s_1 y_1 + s_2 y_2 + \dots + s_\Theta y_\Theta \approx \frac{1}{p}.$$

Mais precisamente, no sentido usado pelo artigo DGHV [2],

$$\left[\sum_i s_i y_i \right]_2 = \frac{1}{p} - \Delta_p, \quad |\Delta_p| < 2^{-\kappa}.$$

Esta construção é implementada no método `gerar_key_esmagada`. O notebook evita representar os y_i como números de vírgula flutuante: guarda apenas os numeradores inteiros u_i e assume sempre o denominador 2^κ . Assim, sempre que se escreve y_i , o programa está na prática a trabalhar com

$$y_i = \frac{u_i}{2^\kappa}.$$

O inteiro secreto original p continua a ser essencial, porque é usado para calcular x_p e, por consequência, para construir os u_i . No entanto, depois desta transformação, a nova fórmula de decifração já não usa diretamente p : usa o vetor s para escolher quais dos valores y_i entram na soma. A chave pública passa então a incluir esta informação auxiliar, enquanto a nova chave secreta é representada por s .

5.2. Estrutura de uma cifra esmagada

A cifra original do DGHV passa a ser designada por c^* . A transformação acrescenta-lhe um vetor auxiliar z , porque a nova decifração precisa de aproximar $\frac{c^*}{p}$ sem dividir diretamente por p .

Na secção anterior construiu-se uma coleção de valores y_i tal que

$$\sum_i s_i y_i \approx \frac{1}{p}.$$

Multiplicando esta aproximação por c^* , obtém-se a ideia que motiva os valores z_i :

$$\sum_i s_i (c^* y_i) \approx \frac{c^*}{p}.$$

Por isso, para cada i , calcula-se $c^* y_i$:

$$z_i = [c^* y_i]_2$$

e conservam-se apenas

$$n = \lceil \log \theta \rceil + 3$$

bits de precisão depois do ponto binário. Assim, z_i é a parte de informação que, quando combinada com o bit secreto s_i , contribui para aproximar $\frac{c^*}{p}$. A nova cifra deixa então de ser só c^* e passa a ser o par

$$(c^*, z).$$

A classe `CifraEsmagada` guarda esse par. O método `calcular_vetor_zs` calcula os z_i representando números racionais por inteiros com uma escala fixa, e `cria_cifra_esmagada` junta c^* ao vetor resultante. Este último método não cifra novamente e não diminui ruído; apenas cria a representação exigida pela Secção 6.1 do artigo DGHV [2].

Quando duas cifras são somadas ou multiplicadas, os métodos `DGHVEsmagado.soma` e `DGHVEsmagado.multiplicar` efetuam a operação sobre os respetivos c^* e recalculam z para o novo inteiro. O homomorfismo continua a vir da componente DGHV c^* ; o vetor z permite que a nova decifração obtenha o mesmo bit sem usar diretamente a divisão por p .

O Lema 6.1 do artigo DGHV [2] é precisamente a garantia de que esta segunda forma de decifrar continua correta: se o ciphertext ainda respeitar os limites de tamanho e de ruído, a decifração esmagada devolve o mesmo bit que a decifração original. Mais formalmente, seja P a operação implementada por `cria_cifra_esmagada`. Enquanto forem satisfeitas as condições desse lema,

$$\text{DecSq}(P(c_1^* + c_2^*)) = \text{DecDGHV}(c_1^* + c_2^*) = m_1 \oplus m_2$$

e

$$\text{DecSq}(P(c_1^* c_2^*)) = \text{DecDGHV}(c_1^* c_2^*) = m_1 \wedge m_2.$$

Portanto, a alteração da chave de decifração não cria um homomorfismo novo nem destrói o anterior. Ela fornece uma segunda forma, aproximadamente equivalente, de decifrar os resultados das mesmas operações inteiras.

5.3. Decifração e condições de correção

A nova decifração é

$$m' = \left[c^* - \left[\sum_i s_i z_i \right] \right]_2.$$

Como $\sum_i s_i y_i \approx \frac{1}{p}$, tem-se

$$\sum_i s_i z_i \approx \frac{c^*}{p}.$$

Assim, o termo arredondado substitui $\left\lfloor \frac{c^*}{p} \right\rfloor$ da decifração original. No notebook, `_dec` calcula a soma usando estes representantes inteiros dos z_i e divide por `escala_z = 2^n` antes de aplicar módulo 2.

O Lema 6.1 do artigo DGHV [2] controla duas fontes de erro. A aproximação de $\frac{1}{p}$ produz o termo $c^* \Delta_p$. Como o artigo garante $|\Delta_p| < 2^{-\kappa}$ e se exige $|c^*| < 2^{\kappa-4}$, tem-se

$$|c^* \Delta_p| < 2^{\kappa-4} \cdot 2^{-\kappa} = 2^{-4} = \frac{1}{16}.$$

A segunda fonte de erro vem do corte de precisão dos z_i . O valor exato seria $[c^* y_i]_2$, mas o notebook guarda apenas n bits depois do ponto binário. Isto acontece em `calcular_vetor_zs`, na linha `z = arredondar_divisao(residuo * self.escala_z, 1 < self.kappa)`. Como `escala_z = 2^n`, o programa guarda um inteiro que representa z_i numa grelha de passo 2^{-n} . O erro de arredondamento de cada z_i fica limitado por metade desse passo; como só θ posições de s são usadas e $n = \lceil \log \theta \rceil + 3$, o erro total também fica pequeno.

As condições verificadas por `condicoes_do_paper` são

$$|c^*| < 2^{\kappa-4}$$

e

$$\text{ruído}(c^*) < \frac{p}{8}.$$

A primeira é a condição usada na conta anterior; a segunda mantém $\frac{c^*}{p}$ próximo de um inteiro. Juntamente com o erro total causado pelo corte de precisão dos z_i , estas margens garantem que o arredondamento seleciona a paridade correta [2].

6. Pipeline do bootstrap implementado

6.1. Chave de bootstrapping

O método `gerar_chave_bootstrap` produz

$$(\text{Enc}(s_1), \dots, \text{Enc}(s_\Theta)).$$

O bootstrap recebe publicamente c^* e z , mas, em vez de usar os bits s_i em claro, usa as suas cifras. Desta forma, avalia a função de decifração sobre valores cifrados. O notebook cifra a própria chave, isto é, os bits de s são cifrados sob a mesma instância do esquema. Tal simplifica a demonstração, mas corresponde a uma hipótese adicional de segurança circular discutida por Gentry [1] e pelo artigo DGHV [2].

6.2. Contagem dos bits por coluna

Cada z_i tem uma representação binária com n bits. Para cada coluna, o método `bootstrap` constrói a lista

```
enc_s if ((z >> coluna) & 1) else zero
```

Se o bit da coluna de z_i for 1, a entrada é $\text{Enc}(s_i)$; se for 0, é uma representação cifrada da constante zero. Isto corresponde a calcular $a_i = s_i z_i$ bit a bit, o primeiro passo da prova da Secção 6.2 [2].

Para perceber a contagem, escreva-se cada z_i em binário. Numa dada posição de bit ℓ , o termo $s_i z_i$ só contribui para essa posição se o bit ℓ de z_i for 1; nesse caso, a contribuição é s_i . Se esse bit for 0, a contribuição é 0. Assim, antes de tratar os transportes entre bits, a quantidade que aparece na

posição ℓ é

$$\sum_i s_i z_{i,\ell},$$

onde $z_{i,\ell}$ é o bit ℓ de z_i . Como todos estes valores são bits, esta soma conta quantos deles são 1. A essa contagem chama-se peso de Hamming da coluna.

O método `bits_peso_coluna` não revela essa contagem; calcula os seus bits homomorficamente. Pelo Lema 6.3 do artigo DGHV [2], o bit j desse peso pode ser escrito como o polinómio simétrico elementar de grau 2^j . A atualização

```
pols[grau] = xor(pols[grau], and_(sigma, pols[grau - 1]))
```

é a recorrência usada para construir esses polinómios. Como apenas θ bits de s são 1, o peso nunca ultrapassa θ e necessita apenas de $\lceil \log(\theta + 1) \rceil$ bits.

6.3. Soma, arredondamento e resultado

Os pesos das colunas são deslocados para a respetiva posição binária e combinados por `somar_bits`. Este método implementa a propagação cifrada do transporte bit a bit:

$$\text{soma} = a \oplus b \oplus \text{carry}$$

e

$$\text{carry novo} = (a \wedge b) \oplus (\text{carry} \wedge (a \oplus b)).$$

O artigo DGHV [2] usa uma técnica *three-for-two* para obter uma análise de profundidade mais favorável. A implementação sequencial do notebook é pedagogicamente mais simples, mas não reproduz exatamente essa parte da prova.

Antes da soma é adicionada a constante 2^{n-1} . Extrair depois o bit de índice n equivale a dividir por 2^n com arredondamento. Finalmente, *bootstrap* calcula

$$\text{Enc} \left(\left\lfloor \sum_i s_i z_i \right\rfloor \bmod 2 \right) \oplus (c^* \bmod 2),$$

obtendo uma cifra de

$$\left[c^* - \left\lfloor \sum_i s_i z_i \right\rfloor \right]_2.$$

Em módulo 2, soma, subtração e XOR coincidem. O resultado é portanto uma nova cifra do mesmo bit que seria devolvido por *_dec*, sem que esse bit tenha sido revelado.

6.4. Avaliação do circuito de teste

O método *avaliar_bootstrap* percorre um circuito em notação pós-fixa. Depois de cada ADD ou MULT, aplica imediatamente *bootstrap*. A pipeline concreta é

```
cifrar entradas
-> aplicar uma porta sobre c_star
-> recalcular z
-> avaliar homomorficamente a decifração
-> obter uma nova cifra do mesmo bit
-> continuar para a porta seguinte
```

O teste do segundo notebook calcula

$$a \wedge b \wedge c \wedge \dots \wedge l$$

com todas as entradas iguais a 1. Sem *refresh*, as multiplicações sucessivas fazem o ruído crescer. Com o *bootstrap* depois de cada porta, o exemplo toy termina com o bit 1 e verifica as duas condições de correção. A implementação completa encontra-se no Anexo B.

7. Limitações da implementação

Os notebooks têm finalidade explicativa. Em particular:

- os parâmetros são demasiado pequenos para segurança real;
- aumentar isoladamente θ torna o circuito de *bootstrap* muito mais caro e pode violar os limites de ruído;
- *const* cria representantes públicos de 0 e 1, não cifras aleatórias destinadas a esconder mensagens;
- o procedimento de combinação usado no *bootstrap* não é o circuito *three-for-two* usado na prova;
- a mesma chave é usada para cifrar os seus próprios bits;
- o objeto Python conserva p para testes, apesar de a descrição transformada tratar s como nova chave secreta;
- o sucesso de uma execução não constitui uma demonstração de segurança nem uma validação dos parâmetros.

Estas diferenças não anulam o valor pedagógico do programa. A estrutura essencial continua visível: a componente c^* preserva XOR e AND, o vetor z permite uma decifração de baixa complexidade, e o *bootstrap* avalia essa decifração usando $\text{Enc}(s)$.

8. Conclusão

O DGHV evidencia de forma particularmente direta como a criptografia homomórfica combina uma estrutura algébrica útil com um termo de erro que protege a mensagem. A soma e a multiplicação de ciphertexts são homomórficas porque preservam a forma $pq + 2r + m$ e, enquanto o ruído permanece controlado, a redução módulo p e módulo 2 recupera a operação correspondente sobre os bits.

Essa mesma multiplicação que permite avaliar portas AND provoca, contudo, o crescimento rápido do ruído. O contributo de Gentry [1] consiste em transformar a decifração numa computação que o próprio esquema consegue avaliar. No DGHV, a decifração é primeiro esmagada: a divisão aproximada por p é substituída por uma soma que usa apenas poucas posições de s e z . Cifrando os bits de s , o bootstrap calcula homomorficamente essa fórmula e produz uma nova cifra do mesmo bit.

Os notebooks anexos mostram esta cadeia de ideias ao nível do código. Não são uma biblioteca criptográfica, mas permitem relacionar cada objeto do artigo DGHV [2] com uma operação concreta e observar tanto a propriedade homomórfica como os limites impostos pelo ruído.

Referências

- [1] C. Gentry, «Fully Homomorphic Encryption Using Ideal Lattices», em *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, em STOC '09. Association for Computing Machinery, 2009, pp. 169–178. doi: 10.1145/1536414.1536440.
- [2] M. van Dijk, C. Gentry, S. Halevi, e V. Vaikuntanathan, «Fully Homomorphic Encryption over the Integers», em *Advances in Cryptology – EUROCRYPT 2010*, em Lecture Notes in Computer Science, vol. 6110. Springer, 2010, pp. 24–43. doi: 10.1007/978-3-642-13190-5_2.
- [3] G. Soares, «Notas sobre Homomorfismos», 2026.

A. Notebook parcialmente homomórfico

Notebook, célula 1 (Markdown)

Este ficheiro esta organizado para ficar parecido com a estrutura da secção 3 do paper:

```
E = (KeyGen, Encrypt, Decrypt, Evaluate)
```

As partes principais são:

```
Parametros:
    Parametros eta, gamma, rho, rho_prime e tau.

SecretKey:
    Guarda a chave secreta p.

PublicKey:
    Guarda x0 e a lista x1, x2, ..., x_tau.

KeyPair:
    Junta a chave secreta e a chave publica.

Ciphertext:
    Guarda o inteiro cifrado.

DGHV:
    Implementa keygen, encrypt, decrypt, evaluate, add e multiply.
```

No artigo, a mensagem cifrada e um bit:

```
m in {0, 1}
```

Por isso, um ciphertext nao e uma string cifrada. E apenas um inteiro grande que cifra um bit. Para cifrar texto, numeros ou strings, deve-se construir uma camada por cima que transforme esses dados em bits e depois cifre bit a bit.

As operacoes homomorficas basicas sao:

```
ADD -> soma modulo 2, ie., XOR

MULT -> multiplicacao modulo 2, ie., AND
```

O metodo evaluate recebe um circuito booleano e aplica essas operacoes sobre ciphertexts.

Nota Importante:
Esta implementacao não é fully homomorphic

Notebook, célula 2 (Python)

```
from secrets import randbelow, randbits

##### utils
class Parametros:
    def __init__(self, eta=96, gamma=384, rho=8, rho_prime=12, tau=32):
        self.eta = eta          # tamanho da chave secreta p
        self.gamma = gamma      # tamanho dos inteiros publicos xi
        self.rho = rho          # ruido usado para gerar a chave publica
        self.rho_prime = rho_prime # ruido usado na cifragem
        self.tau = tau          # quantidade de inteiros na chave publica

    def __str__(self):
        return f"Parametros(eta={self.eta}, gamma={self.gamma}, rho={self.rho}, rho_prime={self.rho_prime}, tau={self.tau})"

class SecretKey:
    def __init__(self, p):
        self.p = p

    def __str__(self):
        return f"SecretKey(p={self.p})"

class PublicKey:
    def __init__(self, x0, xs):
        self.x0 = x0
        self.xs = xs

    def __str__(self):
        return f"PublicKey(x0={self.x0}, xs={self.xs})"
```

```

class KeyPair:
    def __init__(self, sk, pk):
        self.sk = sk
        self.pk = pk

    def __str__(self):
        return f"Secret Key: {self.sk}\nPublic key: {self.pk}"

class Ciphertext:
    def __init__(self, value):
        self.value = value

    def __str__(self):
        return f"Ciphertext(value={self.value})"

#####

class DGHV:
    def __init__(self, params=Parametros()):
        self.params = params
        self.keys = self.keygen()

    def keygen(self):
        p = random_impar(self.params.eta)

        while True:
            public_values = []

            for _ in range(self.params.tau + 1):
                public_values.append(
                    quase_multiplo(
                        p,
                        self.params.gamma,
                        self.params.rho,
                    )
                )

            public_values.sort(reverse=True)
            x0 = public_values[0]
            xs = public_values[1:]

            # Condicao usada no artigo:
            # reiniciar se x0 nao for impar ou se o ruido de x0 nao for par.
            if x0 % 2 == 1 and mod_perto_0(x0, p) % 2 == 0:
                return KeyPair(
                    sk=SecretKey(p=p),
                    pk=PublicKey(x0=x0, xs=xs),
                )

    def enc(self, bit):
        if bit not in (0, 1):
            raise ValueError("Algo de errado não está certo, DGHV cifra apenas bits: 0 ou 1")

        r = random_integer_positivo(self.params.rho_prime)
        subset_sum = 0

        for x in self.keys.pk.xs:
            if randbelow(2) == 1:
                subset_sum += x

        c = bit + 2 * r + 2 * subset_sum
        c = mod_perto_0(c, self.keys.pk.x0)

        return Ciphertext(c)

    def dec(self, ciphertext):
        return mod_perto_0(ciphertext.value, self.keys.sk.p) % 2

    def evaluate(self, circuit, ciphertexts):
        stack = []

        for token in circuit:
            # if token in ciphertexts:
            #     print(ciphertexts[token].value)
            if token == "ADD":
                right = stack.pop()
                left = stack.pop()
                stack.append(self.soma(left, right))
            elif token == "MULT":
                right = stack.pop()
                left = stack.pop()
                stack.append(self.multiplicacao(left, right))
            else:
                stack.append(ciphertexts[token])

```

```

        if len(stack) != 1:
            raise ValueError("circuito invalido")

        return stack[0]

def soma(self, left, right):
    return Ciphertext(left.value + right.value)

def multiplicacao(self, left, right):
    return Ciphertext(left.value * right.value)

def __str__(self):
    return f"Parametros: {self.params}\nKey: {self.keys}"

def random_impar(bits):
    return (1 << (bits - 1)) | randbits(bits - 1) | 1

def random_integer_positivo(bits):
    bound = 1 << bits
    return randbelow(2 * bound - 1) - (bound - 1)

def quase_multiplo(p, gamma, rho):
    q_bound = max(1, (1 << gamma) // p)
    q = randbelow(q_bound)
    r = random_integer_positivo(rho)
    return p * q + r

def mod_perto_0(value, modulus):
    remainder = value % modulus

    if remainder > modulus // 2:
        remainder -= modulus

    return remainder

if __name__ == "__main__":
    Cif = DGHV()

    bits = {
        "a": 1,
        "b": 0,
        "c": 1,
        "d": 1,
        "e": 0,
        "f": 1,
        "g": 1,
        "h": 0,
        "i": 1,
        "j": 1,
    }

    ciphertexts = {}
    for nome, bit in bits.items():
        ciphertexts[nome] = Cif.enc(bit)
        # print(Cif.enc(bit))

    # Circuito booleano:
    # ((a XOR b) AND (c XOR d))
    # XOR ((e AND f) XOR (g AND (h XOR i)))
    # XOR (j AND (a XOR e))
    circuit = [
        "a", "b", "ADD",
        "c", "d", "ADD",
        "MULT",
        "e", "f", "MULT",
        "g", "h", "i", "ADD", "MULT",
        "ADD",
        "ADD",
        "j", "a", "e", "ADD", "MULT",
        "ADD",
    ]

    result = Cif.evaluate(
        circuit=circuit,
        ciphertexts=ciphertexts,
    )

    esperado = (
        (((bits["a"] ^ bits["b"]) & (bits["c"] ^ bits["d"])))
        ^ (((bits["e"] & bits["f"]) ^ (bits["g"] & (bits["h"] ^ bits["i"])))

```

```

    ^ (bits["j"] & (bits["a"] ^ bits["e"])))
)
print("Cifra:", result)
print("resultado esperado:", esperado)
print("resultado decifrado:", Cif.dec(result))

print("\n\n\n")
params_fracos = Parametros(eta=8, gamma=16, rho=6, rho_prime=6, tau=4)

Cif_2 = DGHV(params_fracos)

bits_2 = {
    "a": 1,
    "b": 1,
    "c": 1,
    "d": 1,
    "e": 1,
    "f": 1,
    "g": 1,
    "h": 1,
    "i": 1,
    "j": 1,
    "k": 1,
    "l": 1,
}

ciphertexts_2 = {}
for nome, bit in bits_2.items():
    ciphertexts_2[nome] = Cif_2.enc(bit)

# Circuito booleano: a AND b AND ... AND l
# Como todas as entradas sao 1, o esperado e 1.
# Com parametros fracos e muitas multiplicacoes, o ruido pode fazer falhar.

circuito_2 = [
    "a", "b", "MULT",
    "c", "MULT",
    "d", "MULT",
    "e", "MULT",
    "f", "MULT",
    "g", "MULT",
    "h", "MULT",
    "i", "MULT",
    "j", "MULT",
    "k", "MULT",
    "l", "MULT",
]

result_falha = Cif_2.evaluate(
    circuit=circuito_2,
    ciphertexts=ciphertexts_2,
)

esperado_falha = 1
decifrado_falha = Cif_2.dec(result_falha)

print("Cifra:", result_falha)
print("resultado esperado:", esperado_falha)
print("resultado decifrado:", decifrado_falha)

```

Cifra:

Ciphertext(value=-25263508477880289220386951808987219303437707001965386197360928826573508585249671702992286946757250240674440200465247240308769

resultado esperado: 0

resultado decifrado: 0

Cifra: Ciphertext(value=8926567939888542614062991487633194777676451502400)

resultado esperado: 1

resultado decifrado: 0

B. Notebook com decifração esmagada e bootstrap

Notebook, célula 1 (Markdown)

DGHV com decifração esmagada

Implementação da transformação da Secção 6.1 do paper: a chave secreta passa a ser um vetor esparso s , a chave pública recebe valores $y_i = u_i / 2^k$, e cada texto cifrado fica com a forma (c_{star}, z) .

A decifração segue a fórmula do paper:

```
```text
m = [c_star - round(sum(s_i * z_i))]_2
```
```

Depois, `bootstrap` avalia homomorficamente esta decifração usando a chave de bootstrapping `Enc(s_i)`. Ou seja, o refresh não chama `decifrar` seguido de `cifrar`.

Nota: os parâmetros do teste são toy para o notebook correr. A construção segue o mecanismo do paper, mas não tem segurança prática.

Notebook, célula 2 (Python)

```
from secrets import randbelow

from somewhat_homomorphic import *
```

Notebook, célula 3 (Python)

```
def ceil_log2(value):
    return max(0, (value - 1).bit_length())

def arredondar_divisao(numerador, denominador):
    if denominador <= 0:
        raise ValueError("denominador deve ser positivo")

    if numerador >= 0:
        return (numerador + denominador // 2) // denominador

    return -((-numerador + denominador // 2) // denominador)

class CifraEsmagada:
    def __init__(self, c_star, zs):
        self.c_star = c_star
        self.zs = zs
        self.value = c_star

    def __str__(self):
        return f"CifraEsmagada(c_star={self.c_star}, zs=<len {len(self.zs)}>)"

class DGHVEsmagado:
    def __init__(self, params=None, k=None, theta=8, big_theta=128):
        if theta <= 0 or big_theta < theta:
            raise ValueError("0 < theta <= big_theta")

        self.params = params or Parametros()
        self.base = DGHV(self.params)
        # k = gamma*eta/rho'
        self.k = k or max(
            self.params.gamma + 8,
            (self.params.gamma * self.params.eta) // max(1, self.params.rho_prime),
        )
        self.theta = theta
        self.big_theta = big_theta
        # n = ceil(log(theta)) + 3
        self.n = ceil_log2(theta) + 3
        # escala_z = 2^n,
        self.escala_z = 1 << self.n
        self.s, self.us = self.gerar_key_esmagada()

    def gerar_key_esmagada(self):
        p = self.base.keys.sk.p
        # u_i em [0, 2^(k+1)); modulo = 2^(k+1)
        modulo = 1 << (self.k + 1)
        # x_p = round(2^k/p)
        x_p = arredondar_divisao(1 << self.k, p)

        posicoes = []
```



```

usadas = set()
while len(posicoes) < self.theta:
    pos = randbelow(self.big_theta)
    if pos not in usadas:
        usadas.add(pos)
        posicoes.append(pos)

# s=(s_1,...,s_Theta), sum_i s_i = theta
s = [1 if i in usadas else 0 for i in range(self.big_theta)]

us = [randbelow(modulo) for _ in range(self.big_theta)]

# sum_i s_i*u_i = x_p mod 2^(k+1)
ultima = posicoes[-1]
soma_parcial = sum(us[i] for i in posicoes[:-1])
us[ultima] = (x_p - soma_parcial) % modulo

return s, us

def calcular_vetor_zs(self, c_star):
    modulo = 1 << (self.k + 1)
    zs = []

    for u in self.us:
        # y_i = u_i/2^k e z_i = [c_star*y_i]_2
        residuo = (c_star * u) % modulo
        z = arredondar_divisao(residuo * self.escala_z, 1 << self.k)
        zs.append(z % (2 * self.escala_z))

    return zs

def cria_cifra_esmagada(self, c_star):
    return CifraEsmagada(c_star, self.calcular_vetor_zs(c_star))

def _enc(self, bit):
    return self.cria_cifra_esmagada(self.base.enc(bit).value)

def _dec(self, cifra):
    # m' = [c_star - round(sum_i s_i*z_i)]_2
    total = sum(si * zi for si, zi in zip(self.s, cifra.zs))
    quociente_aproximado = arredondar_divisao(total, self.escala_z)
    return (cifra.c_star - quociente_aproximado) % 2

def const(self, bit):
    return self.cria_cifra_esmagada(bit & 1)

def xor(self, esquerda, direita):
    return self.soma(esquerda, direita)

def and_(self, esquerda, direita):
    return self.multiplicar(esquerda, direita)

def gerar_chave_bootstrap(self):
    # chave_bootstrap = (Enc(s_1),...,Enc(s_Theta))
    self.chave_bootstrap = [self.enc(bit) for bit in self.s]
    return self.chave_bootstrap

def bits_peso_coluna(self, entradas):
    # bit j do peso de Hamming = e_(2^j)(sigma) mod 2
    num_bits = ceil_log2(self.theta + 1)
    max_grau = 1 << (num_bits - 1)
    pols = [self.const(0) for _ in range(max_grau + 1)]
    pols[0] = self.const(1)

    for sigma in entradas:
        for grau in range(max_grau, 0, -1):
            # e_g <- e_g + sigma*e_(g-1) mod 2
            pols[grau] = self.xor(pols[grau], self.and_(sigma, pols[grau - 1]))

    return [pols[1 << bit] for bit in range(num_bits)]

def somar_bits(self, esquerda, direita):
    # soma = a xor b xor carry; carry'=(a and b) xor (carry and (a xor b))
    carry = self.const(0)
    resultado = []

    for a, b in zip(esquerda, direita):
        a_xor_b = self.xor(a, b)
        resultado.append(self.xor(a_xor_b, carry))
        carry = self.xor(self.and_(a, b), self.and_(carry, a_xor_b))

    resultado.append(carry)
    return resultado

```

```

def bootstrap(self, cifra):
    # if not self.chave_bootstrap:
    #     self.gerar_chave_bootstrap()

    bits_contador = ceil_log2(self.theta + 1)
    largura = self.n + bits_contador + 2
    # somar 2^(n-1) antes de extrair o bit n implementa arredondamento por 2^n
    arredondamento = 1 << (self.n - 1)
    total = [self.const((arredondamento >> j) & 1) for j in range(largura)]
    zero = self.const(0)

    for coluna in range(self.n + 1):
        # a_i = s_i * z_i
        entradas = [
            enc_s if ((z >> coluna) & 1) else zero
            for enc_s, z in zip(self.chave_bootstrap, cifra.zs)
        ]
        peso = self.bits_peso_coluna(entradas)
        termo = [zero for _ in range(largura)]

        for bit, enc_bit in enumerate(peso):
            if coluna + bit < largura:
                termo[coluna + bit] = enc_bit

        total = self.somar_bits(total, termo)[:largura]

    # Enc(round(sum_i s_i * z_i mod 2) xor (c_star mod 2))
    bit_quociente = total[self.n]
    return self.xor(bit_quociente, self.const(cifra.c_star & 1))

# def evaluate(self, circuit, ciphertexts):
#     stack = []
#     for token in circuit:
#         # if token in ciphertexts:
#         #     print(ciphertexts[token].value)
#         if token == "ADD":
#             right = stack.pop()
#             left = stack.pop()
#             stack.append(self.soma(left, right))
#         elif token == "MULT":
#             right = stack.pop()
#             left = stack.pop()
#             stack.append(self.multiplicacao(left, right))
#         else:
#             stack.append(ciphertexts[token])
#     if len(stack) != 1:
#         raise ValueError("circuito invalido")
#     return stack[0]

def avaliar_bootstrap(self, circuito, cifras):
    stack = []

    for token in circuito:
        if token == "ADD":
            direita = stack.pop()
            esquerda = stack.pop()
            stack.append(self.bootstrap(self.soma(esquerda, direita)))
        elif token == "MULT":
            direita = stack.pop()
            esquerda = stack.pop()
            stack.append(self.bootstrap(self.multiplicar(esquerda, direita)))
        else:
            stack.append(cifras[token])

    if len(stack) != 1:
        raise ValueError("circuito inválido")

    return stack[0]

def soma(self, esquerda, direita):
    # DecSq(P(c1*c2)) = m1 xor m2
    return self.cria_cifra_esmagada(esquerda.c_star + direita.c_star)

def multiplicar(self, esquerda, direita):
    # DecSq(P(c1*c2)) = m1 and m2
    return self.cria_cifra_esmagada(esquerda.c_star * direita.c_star)

def decifrar_original_para_teste(self, cifra):
    return self.base.dec(Ciphertext(cifra.c_star))

def condicoes_do_papel(self, cifra):
    # |c_star| < 2^(k-4) e ruido(c_star) < p/8
    p = self.base.keys.sk.p
    ruido = abs(mod_perto_0(cifra.c_star, p))

```

```

        return {
            "|c_star| < 2^(k-4)": abs(cifra.c_star) < (1 << (self.k - 4)),
            "ruído < p/8": 8 * ruído < p,
        }

# enc = _enc
# dec = _dec
# # evaluate = avaliar

def __str__(self):
    return (
        f"DGHVEsmagado(k={self.k}, theta={self.theta}, "
        f"big_theta={self.big_theta}, n={self.n})"
    )

```

Notebook, célula 4 (Python)

```

params = Parametros(eta=160, gamma=640, rho=8, rho_prime=12, tau=32)
Cif_2 = DGHVEsmagado(params, theta=2, big_theta=4)
Cif_2.gerar_chave_bootstrap()

bits_2 = {
    "a": 1,
    "b": 1,
    "c": 1,
    "d": 1,
    "e": 1,
    "f": 1,
    "g": 1,
    "h": 1,
    "i": 1,
    "j": 1,
    "k": 1,
    "l": 1,
}

circuito_2 = [
    "a", "b", "MULT",
    "c", "MULT",
    "d", "MULT",
    "e", "MULT",
    "f", "MULT",
    "g", "MULT",
    "h", "MULT",
    "i", "MULT",
    "j", "MULT",
    "k", "MULT",
    "l", "MULT",
]

cifras_2 = {nome: Cif_2.enc(bit) for nome, bit in bits_2.items()}
resultado_2 = Cif_2.avaliar_bootstrap(circuito_2, cifras_2)
esperado_2 = 1
obtido_2 = Cif_2._dec(resultado_2)

print(Cif_2)
print("peso da chave secreta s:", sum(Cif_2.s))
print("tamanho da chave de bootstrapping:", len(Cif_2.chave_bootstrap))
print("resultado esperado:", esperado_2)
print("resultado com bootstrapping:", obtido_2)
print("condições finais do paper:", Cif_2.condicoes_do_paper(resultado_2))

assert obtido_2 == esperado_2

```

```

DGHVEsmagado(k=8533, theta=2, big_theta=4, n=4)
peso da chave secreta s: 2
tamanho da chave de bootstrapping: 4
resultado esperado: 1
resultado com bootstrapping: 1
condições finais do paper: {'|c_star| < 2^(k-4)': True, 'ruído < p/8': True}

```